Day 17: K-Means Clustering

Unsupervised Learning — Grouping Data Without Labels

Sandesh Dhital

Nesfield International College

August 9, 2026

Supervised vs Unsupervised Learning

Supervised Learning

- Data has **labels** (answers)
- Goal: predict the label
- Examples: Linear Regression, KNN, Decision Tree

Unsupervised Learning

- Data has **no labels**
- Goal: find hidden **patterns/groups**
- Examples: K-Means, DBSCAN, PCA

K-Means Clustering is the most popular unsupervised algorithm — it groups similar data points into K clusters.

What is K-Means Clustering?

K-Means partitions n data points into K clusters, where each point belongs to the cluster with the **nearest centroid** (mean).

Key Concepts:

- **K** = number of clusters (you choose this!)
- **Centroid** = the center (mean) of a cluster
- **Assignment** = each point goes to its nearest centroid
- **Update** = recalculate centroids after assignment

Objective

Minimize the **Within-Cluster Sum of Squares (WCSS)**: $J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$ where μ_k is the centroid of cluster C_k.

The K-Means Algorithm — Steps

1. **Choose K** — decide how many clusters you want
2. **Initialize** — randomly pick K points as initial centroids
3. **Assign** — assign each data point to the nearest centroid (using Euclidean distance)
4. **Update** — recalculate centroids as the mean of all points in each cluster
5. **Repeat** steps 3–4 until centroids stop moving (convergence)

Distance formula (Euclidean):

$$d(x, \mu) = \sqrt{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2 + \cdots + (x_n - \mu_n)^2}$$

Convergence

The algorithm converges when assignments no longer change between iterations.

Step-by-Step Example: The Dataset

We have 6 data points in 2D. Goal: cluster them into $K = 2$ groups.

Point	x_1	x_2
A	1	1
B	2	1
C	1	2
D	6	6
E	7	6
F	6	7

Step 1: We choose $K = 2$ (we want 2 clusters).
Step 2: Randomly select initial centroids: $\mu_1 = A = (1, 1)$ and $\mu_2 = D = (6, 6)$

Iteration 1: Calculate Distances to Each Centroid

Centroids: $\mu_1 = (1, 1), \quad \mu_2 = (6, 6)$

Point	(x_1, x_2)	$d(\mu_1)$	$d(\mu_2)$	**Cluster**
A	(1, 1)	$\sqrt{0+0} = 0$	$\sqrt{25+25} = 7.07$	C_1
B	(2, 1)	$\sqrt{1+0} = 1$	$\sqrt{16+25} = 6.40$	C_1
C	(1, 2)	$\sqrt{0+1} = 1$	$\sqrt{25+16} = 6.40$	C_1
D	(6, 6)	$\sqrt{25+25} = 7.07$	$\sqrt{0+0} = 0$	C_2
E	(7, 6)	$\sqrt{36+25} = 7.81$	$\sqrt{1+0} = 1$	C_2
F	(6, 7)	$\sqrt{25+36} = 7.81$	$\sqrt{0+1} = 1$	C_2

Result: $C_1 = \{A, B, C\}, \quad C_2 = \{D, E, F\}$

Iteration 1: Update Centroids

Recalculate centroids as the **mean** of all points in each cluster.

Cluster 1: $\{A(1,1), B(2,1), C(1,2)\}$

$$\mu_1^{\text{new}} = \left(\frac{1+2+1}{3},\ \frac{1+1+2}{3}\right) = \left(\frac{4}{3},\ \frac{4}{3}\right) = (1.33,\ 1.33)$$

Cluster 2: $\{D(6,6), E(7,6), F(6,7)\}$

$$\mu_2^{\text{new}} = \left(\frac{6+7+6}{3},\ \frac{6+6+7}{3}\right) = \left(\frac{19}{3},\ \frac{19}{3}\right) = (6.33,\ 6.33)$$

New centroids: $\mu_1 = (1.33, 1.33), \quad \mu_2 = (6.33, 6.33)$
Centroids have moved! We need another iteration.

Iteration 2: Reassign Points to New Centroids

New centroids: $\mu_1 = (1.33, 1.33)$, $\mu_2 = (6.33, 6.33)$

Point	(x_1, x_2)	$d(\mu_1)$	$d(\mu_2)$	**Cluster**
A	(1, 1)	$\sqrt{0.11 + 0.11} = 0.47$	$\sqrt{28.41 + 28.41} = 7.54$	C_1
B	(2, 1)	$\sqrt{0.45 + 0.11} = 0.75$	$\sqrt{18.77 + 28.41} = 6.87$	C_1
C	(1, 2)	$\sqrt{0.11 + 0.45} = 0.75$	$\sqrt{28.41 + 18.77} = 6.87$	C_1
D	(6, 6)	$\sqrt{21.81 + 21.81} = 6.60$	$\sqrt{0.11 + 0.11} = 0.47$	C_2
E	(7, 6)	$\sqrt{32.15 + 21.81} = 7.35$	$\sqrt{0.45 + 0.11} = 0.75$	C_2
F	(6, 7)	$\sqrt{21.81 + 32.15} = 7.35$	$\sqrt{0.11 + 0.45} = 0.75$	C_2

Same assignments as Iteration 1! $\Rightarrow$ Algorithm has **converged**.

Final Clustering Result

Cluster 1 (centroid: 1.33, 1.33)

- A (1, 1)
- B (2, 1)
- C (1, 2)

Points in the **lower-left** region.

Cluster 2 (centroid: 6.33, 6.33)

- D (6, 6)
- E (7, 6)
- F (6, 7)

Points in the **upper-right** region.

WCSS Calculation:

$$
\begin{aligned}
J &= \sum_{C_1} \|x_i - \mu_1\|^2 + \sum_{C_2} \|x_i - \mu_2\|^2 \\
&= (0.22 + 0.56 + 0.56) + (0.22 + 0.56 + 0.56) \\
&= 1.34 + 1.34 = \boxed{2.68}
\end{aligned}
$$

How to Choose K? The Elbow Method

We don't always know the right value of K. The **Elbow Method** helps:

1. Run K-Means for $K = 1, 2, 3, 4, \ldots$
2. For each K, record the **WCSS** (inertia)
3. Plot K vs WCSS
4. Look for the "elbow" — where WCSS stops decreasing sharply

Intuition

- $K = 1$: one big cluster $\rightarrow$ very high WCSS
- $K = n$: each point is its own cluster $\rightarrow$ WCSS $= 0$
- The **elbow point** balances simplicity and accuracy

After the elbow, adding more clusters gives **diminishing returns**.

Important Properties of K-Means

- **Must specify K** in advance
- **Sensitive to initialization** — different starting centroids can give different results
- **K-Means++**: smart initialization that spreads initial centroids apart (default in scikit-learn)
- **Always converges**, but may find a local minimum (not global)
- **Assumes spherical clusters** of similar size
- **Sensitive to outliers** — outliers pull centroids away

K-Means++ Initialization

Instead of random centroids:

1. Pick first centroid randomly
2. Pick next centroid with probability proportional to d^2 from nearest existing centroid
3. Repeat until K centroids are chosen

Real-World Applications of K-Means

Domain	Application
Marketing	Customer segmentation
Image Processing	Color quantization / compression
Biology	Gene expression grouping
Document Analysis	Topic clustering
Anomaly Detection	Identify outliers far from any centroid
Geography	Grouping locations (e.g., delivery zones)

Example

An e-commerce company uses K-Means to segment customers into groups like "budget shoppers," "frequent buyers," and "premium customers" based on purchase behavior.

K-Means: Pros and Cons

Advantages

- Simple and fast
- Scales well to large datasets
- Easy to interpret results
- Works well for spherical clusters
- Guaranteed convergence

Disadvantages

- Must choose K in advance
- Sensitive to initialization
- Assumes equal-sized spherical clusters
- Sensitive to outliers
- Cannot find non-convex clusters

Alternatives: DBSCAN (no need to specify K, finds arbitrary shapes), Hierarchical Clustering (dendrograms), Gaussian Mixture Models (soft assignments).

Summary

1. K-Means is an **unsupervised** algorithm that groups data into K clusters
2. It works by iteratively **assigning points** to nearest centroids and **updating centroids**
3. Uses **Euclidean distance** to measure similarity
4. Minimizes **WCSS** (Within-Cluster Sum of Squares)
5. Use the **Elbow Method** to find the best K
6. **K-Means++** provides better initialization
7. Converges when cluster assignments stop changing

Next Steps

Practice with the notebook! Try different values of K and observe how clusters change.